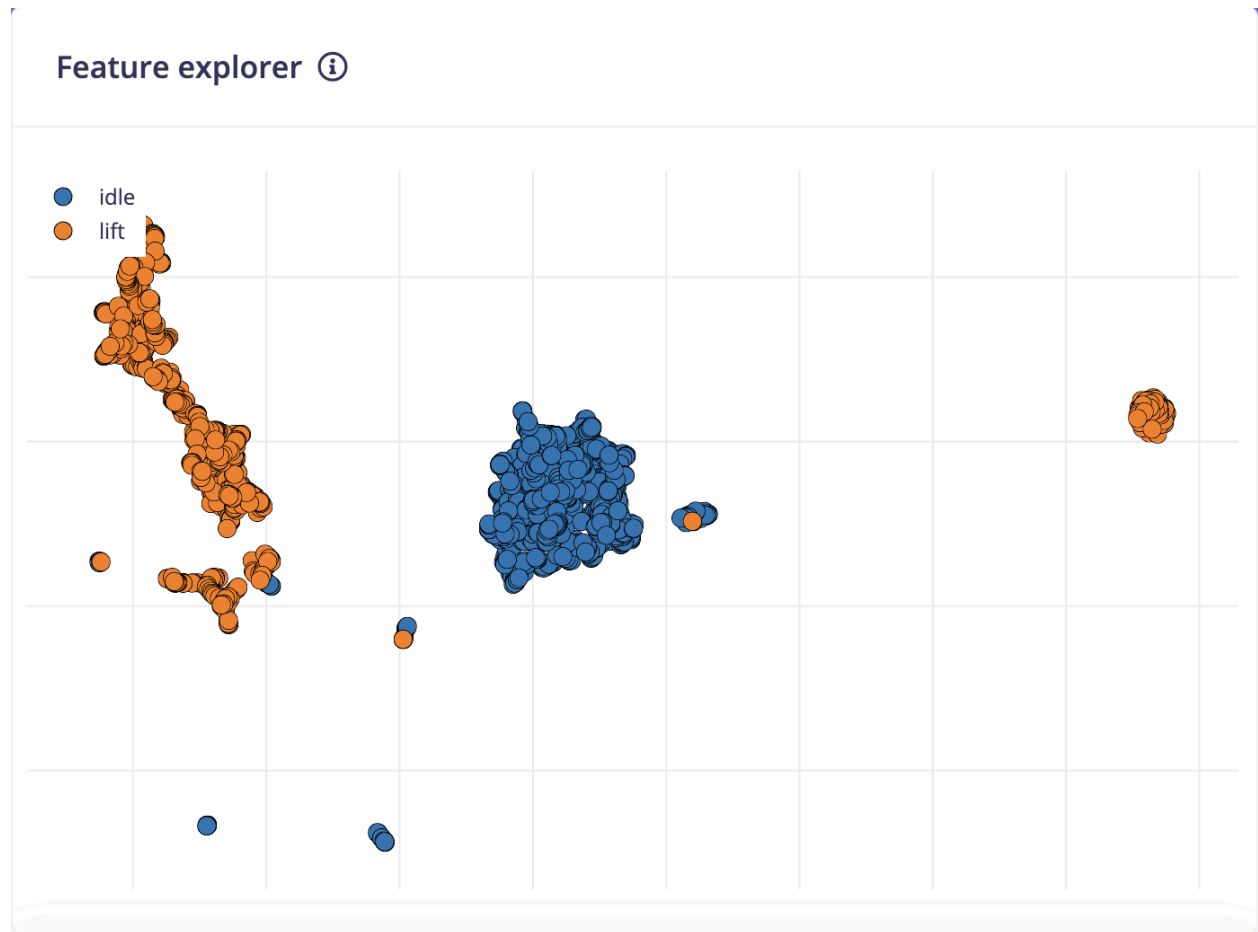


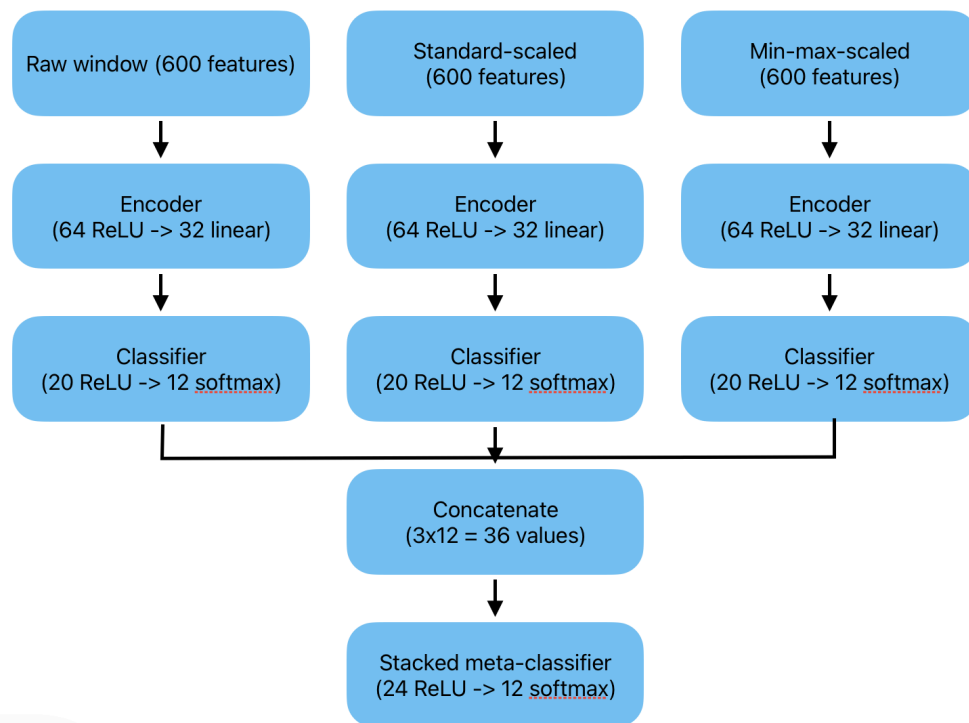
Part 1:

<https://studio.edgeimpulse.com/public/1082683/live>



Part 2:

Question 1:



Question 2:

The notebook compresses each model in three stages. The first stage is pruning which uses a PolynomialDecay schedule that increases sparsity from 10 to 80% during training while zeroing out the least important weights over time. The pruning layer is then stripped before QAT since quantization needs to add its own. Then it is QAT where it fine-tunes the model while simulating int8 rounding so its weights stay accurate after the quantization. Ordinary training could un-prune weights during QAT, so a custom mask-enforcement callback re-zeroes the pruned weights after every batch and epoch to keep the sparsity. Then the model is converted to an int TFLite model using a small representative dataset that lets the converter calculate correct scale and zero-point values for accurate quantization. This combo matters for deployment because the Arduino board has limited RAM, so pruning will reduce the number of meaningful parameters, while the int8 quantization will shrink the model size and make it so it is able to run way faster on the device.

Question 3:

```
Output Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem2101') New Line 115200 baud
-----
22:32:36.376 -> Samples collected: 50
22:32:36.904 -> Samples collected: 75
22:32:37.430 -> Samples collected: 100
22:32:37.430 -> Window collected. Running ensemble inference...
22:32:37.462 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
22:32:37.462 -> Standard-scaled model scores: 0.0000, 0.0508, 0.2070, 0.0078, 0.0391, 0.0000, 0.0195, 0.0000, 0.0000, 0.0078, 0.0078, 0.666
22:32:37.495 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.0820, 0.1016, 0.1758, 0.0039, 0.5391, 0.0000, 0.0000, 0.0000, 0.0000, 0.0937
22:32:37.495 -> Meta-classifier scores: 0.0039, 0.1016, 0.4297, 0.0039, 0.0039, 0.0156, 0.1016, 0.0039, 0.0039, 0.0039, 0.0039, 0.3281
22:32:37.495 -> -----
22:32:37.495 -> Final prediction: Lying down
22:32:37.495 -> Class index: 2
22:32:37.495 -> Confidence score: 0.4297
22:32:37.495 -> -----
22:32:38.022 -> Samples collected: 25
22:32:38.550 -> Samples collected: 50
22:32:39.075 -> Samples collected: 75
22:32:39.632 -> Samples collected: 100
22:32:39.632 -> Window collected. Running ensemble inference...
22:32:39.664 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
22:32:39.664 -> Standard-scaled model scores: 0.0000, 0.0430, 0.2070, 0.0078, 0.0391, 0.0000, 0.0195, 0.0000, 0.0000, 0.0078, 0.0078, 0.666
22:32:39.664 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.0820, 0.1016, 0.1758, 0.0039, 0.5391, 0.0000, 0.0000, 0.0000, 0.0000, 0.0937
22:32:39.664 -> Meta-classifier scores: 0.0039, 0.1016, 0.4492, 0.0039, 0.0039, 0.0156, 0.0937, 0.0039, 0.0039, 0.0039, 0.0039, 0.3203
22:32:39.664 -> -----
22:32:39.664 -> Final prediction: Lying down
22:32:39.664 -> Class index: 2
22:32:39.664 -> Confidence score: 0.4492
22:32:39.664 -> -----
22:32:40.192 -> Samples collected: 25
```

Looking at the serial output, the output of “lying down” is appropriate as the board is laying still on a flat surface. I do notice that if the board is oriented in two different directions it will say the prediction is “jump front and back”, but the other two directions will show “lying down”. I think a change would be to be more accurate with the axis coordinates to make sure that there is less of a mistake when the code observes the numbers, and for “jump front and back”, there should be movement.

Question 4:

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

The model was pretty accurate considering that there wasn't much time between each check for the actions happening, but sometimes when the board is on a flat surface at a different angle, the serial monitor will give a wrong action instead of the "lying down" action.

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

I think the sensor placement and the axis orientation would be factors in the board miscalculating the action of the board. If the board is in a different orientation than when it was originally tested in Edge Impulse, the code might not recognize that and try to find the next closest action to it.